

# OrderFlow: Event-Driven Order Processing System

**Project Report** — An advanced order-processing application demonstrating event-driven architecture using LMAX Disruptor, Spring Boot, and React

## Project Overview

OrderFlow is an event-driven order processing system designed to handle order intake, processing, and status updates with high throughput and low latency. It demonstrates how a modern architecture can improve responsiveness, scalability, and reliability across the order lifecycle.

## Key Achievements

- Delivered the core event-driven order workflow from submission to completion
- Integrated LMAX Disruptor for efficient asynchronous event handling
- Built a Spring Boot backend for orchestration and business logic
- Created a React-based interface for order visibility and interaction
- Established a foundation for scalable, maintainable order automation

## Project Duration

The project was completed over the course of a focused development cycle, from initial architecture planning through implementation and integration testing.

## Team Size

Developed by a small, cross-functional team working collaboratively across backend, frontend, and system architecture responsibilities.

## Impact

OrderFlow showcases how event-driven design can streamline order operations, improve system throughput, and support a more resilient user experience for modern commerce workflows.

# Project Scope: Defining OrderFlow's Core

A robust event-driven system demands clear functional and non-functional specifications to guide development and ensure success.

## Functional Requirements

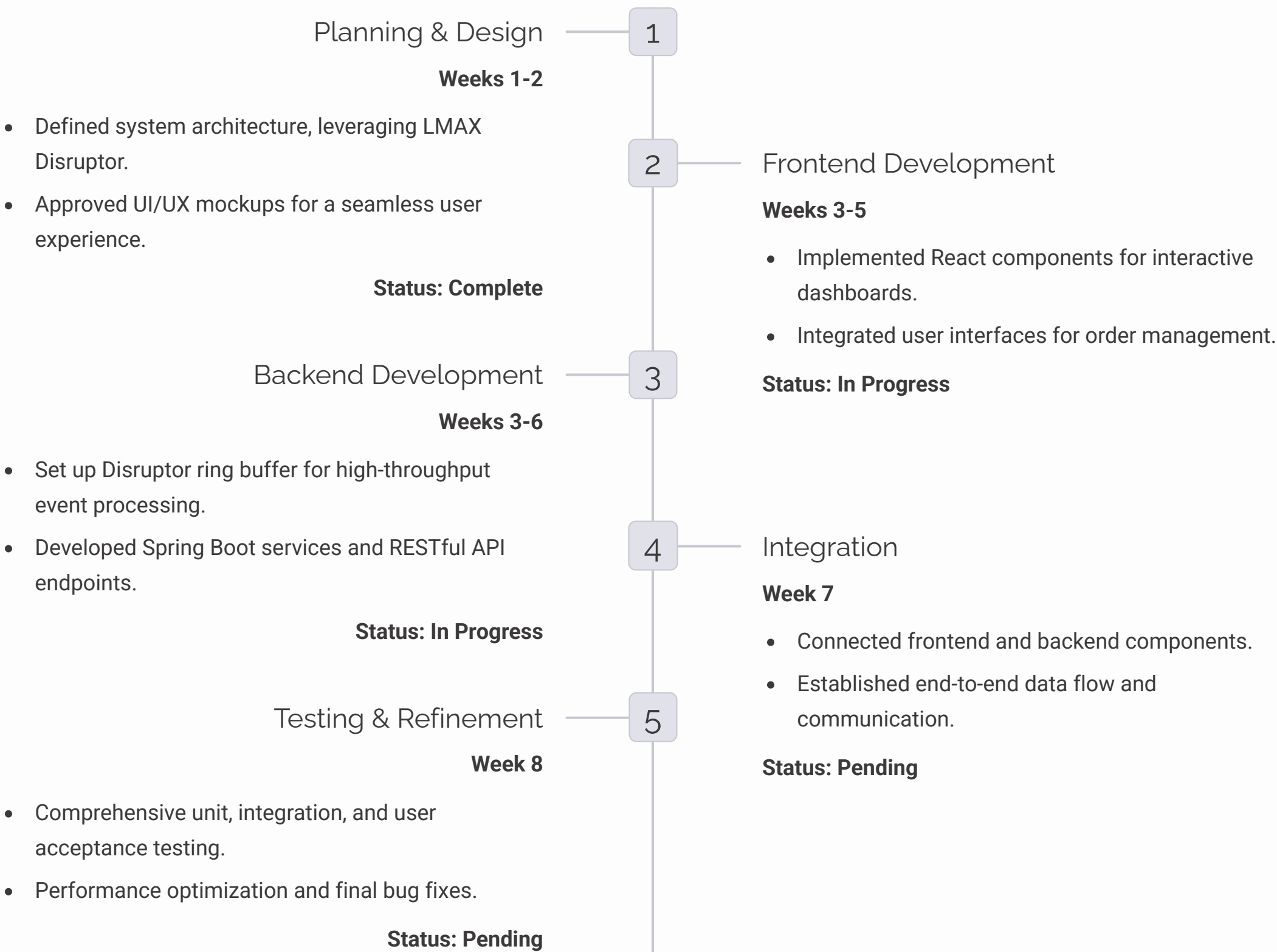
- **Order Submission:** Secure ingestion of new orders from multiple channels, including validation and initial state assignment.
- **Order Retrieval:** Efficient querying of order status, history, and details for customer service and operational teams.
- **Event Processing:** Real-time handling of order lifecycle events (e.g., payment, fulfillment) via the LMAX Disruptor.
- **Real-time Updates:** Instantaneous propagation of status changes to all relevant user interfaces and integrated services.

## Non-Functional Requirements

- **Performance:** Process 5,000 orders/second with <100ms latency for critical operations.
- **Scalability:** Support horizontal scaling to accommodate peak loads up to 10,000 orders/second.
- **Reliability:** Maintain 99.99% uptime with robust error handling and fault tolerance.
- **Security:** Implement end-to-end encryption, access controls, and compliance with data privacy regulations.

# Project Timeline: OrderFlow Development

Our project is structured into distinct phases to ensure systematic progress and timely delivery of the event-driven order processing system.



# Executive Summary

OrderFlow is an event-driven order processing platform designed to deliver fast, reliable, and maintainable full-stack workflows for modern commerce operations. The project combines a responsive React/Vite frontend with a Java Spring Boot backend, using the LMAX Disruptor to power high-throughput asynchronous event handling across the system.

Built by a collaborative five-person team, the application brings together frontend development, backend services, API testing, and integration work into a cohesive end-to-end solution. Core functionality is already operational, with real-time order handling, REST APIs, and the event-driven backend in place, while final refinements continue to improve polish and performance.

## Full-Stack Event-Driven System

React/Vite frontend paired with a Java Spring Boot backend for seamless end-to-end order handling.

## LMAX Disruptor at the Core

High-performance lock-free ring buffer architecture enabling asynchronous, high-throughput event processing.

## Integrated & Operational

Core functionality is complete — REST APIs, event-driven backend, and real-time order handling are all running. Minor refinements in progress.

## 5-Person Collaborative Team

Frontend, backend, API testing, and integration handled across a cross-functional team of five developers.

# Problem Statement

1

## Challenge

Synchronous request-response patterns create bottlenecks as order volume scales.

2

## Limitation

Direct processing inside request handlers reduces responsiveness and hinders scaling.

3

## Gap

No clear separation between order producers (frontend) and order consumers (backend processors).

4

## Solution

LMAX Disruptor decouples submission from processing — enabling asynchronous, high-throughput order handling.

# Project Objectives

## 1 React/Vite Frontend Dashboard

Professional order submission and management interface.

## 2 Spring Boot REST APIs

Receive and coordinate order requests across the system.

## 3 LMAX Disruptor Event Processing

Ring buffer architecture for lock-free, high-performance throughput.

## 4 Frontend-Backend Integration

Seamless REST communication bridging both layers.

## 1 Async Design Patterns

Demonstrate event-driven design for asynchronous order processing.

## 2 API Testing with Postman

Comprehensive endpoint validation across all request scenarios.

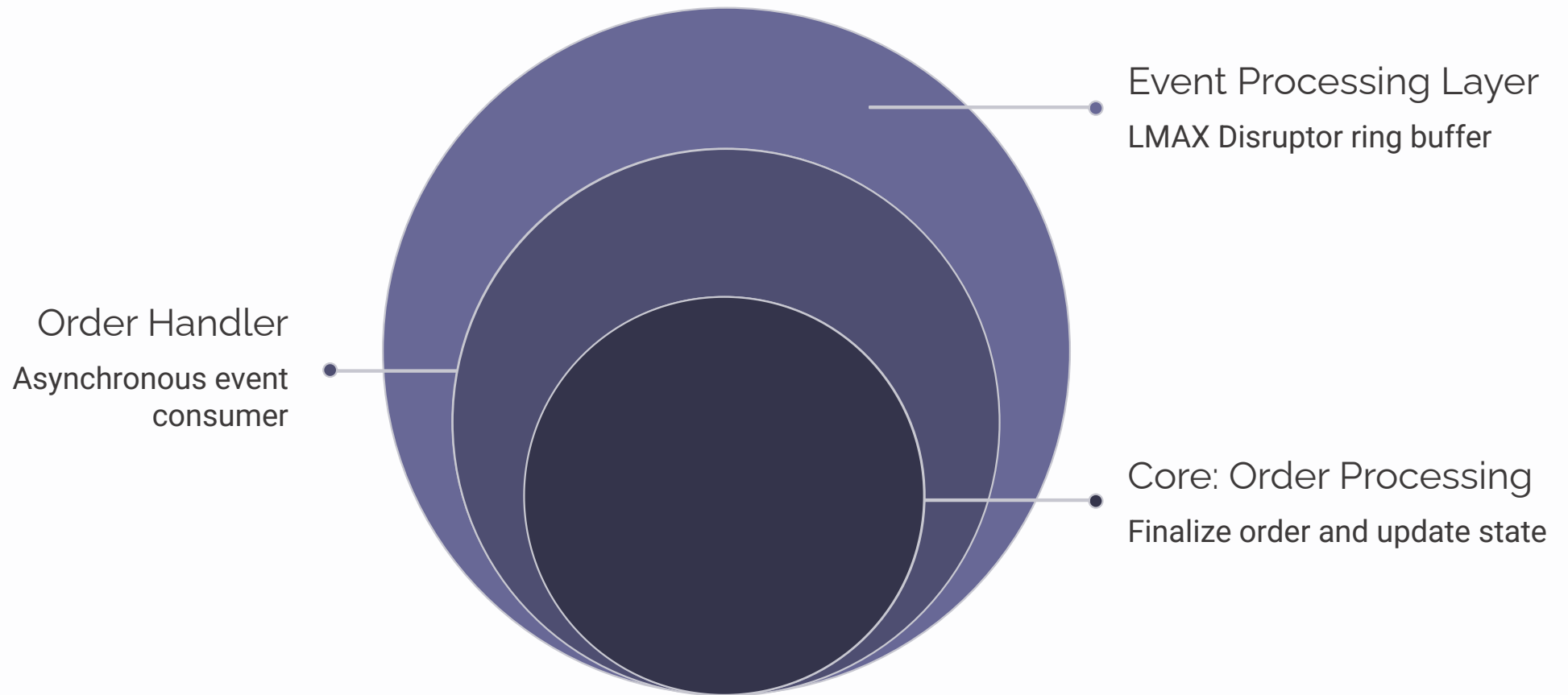
## 3 Git/GitHub Version Control

Collaborative development with proper branching and commit practices.

## 4 Full Documentation

Architecture, implementation, and testing procedures fully documented.

# System Architecture Overview



Data flows from the user through the React frontend, across REST APIs into Spring Boot services, then into the LMAX Disruptor ring buffer — where lock-free event handlers asynchronously complete each order. Git/GitHub and Postman support the full development lifecycle.

# Technology Stack



## Frontend

React 18+, Vite build tool, JavaScript/CSS, responsive UI components.



## Backend

Java 11+, Spring Boot 2.7+, Spring Web REST framework for API handling.



## Event Processing

LMAX Disruptor — lock-free ring buffer for high-throughput concurrency.



## Communication

REST APIs over HTTP, JSON data format for all request and response payloads.



## Testing

Postman for API validation, manual integration and end-to-end testing.



## Version Control

Git and GitHub for collaborative development, branching, and history management.



# Frontend Implementation

## Component-Based React Application

Built on Vite with hot module replacement (HMR) for rapid iterative development.

- **Dashboard UI:** Navigation, order forms, and real-time status displays in a professional layout
- **Order Entry Form:** Buy/Sell toggle, Limit/Market type, Price and Quantity inputs
- **State Management:** React hooks — `useState` and `useEffect` — for component state and side effects
- **API Integration:** Axios/Fetch for asynchronous HTTP communication with the Spring Boot backend
- **Responsive Styling:** CSS-based design ensuring usability across device sizes

# Backend Implementation — Spring Boot Services

## REST Controllers

Endpoints for order submission, retrieval, and status management exposed via Spring Web.

## Service Layer

Business logic for order validation, processing coordination, and event publishing to the Disruptor.

## OrderEvent Class

Data structure carrying order ID, type, price, quantity, and timestamp through the ring buffer.

## Event Producer

Obtains sequence slots, populates `OrderEvent` objects, and publishes them to the Disruptor ring buffer.

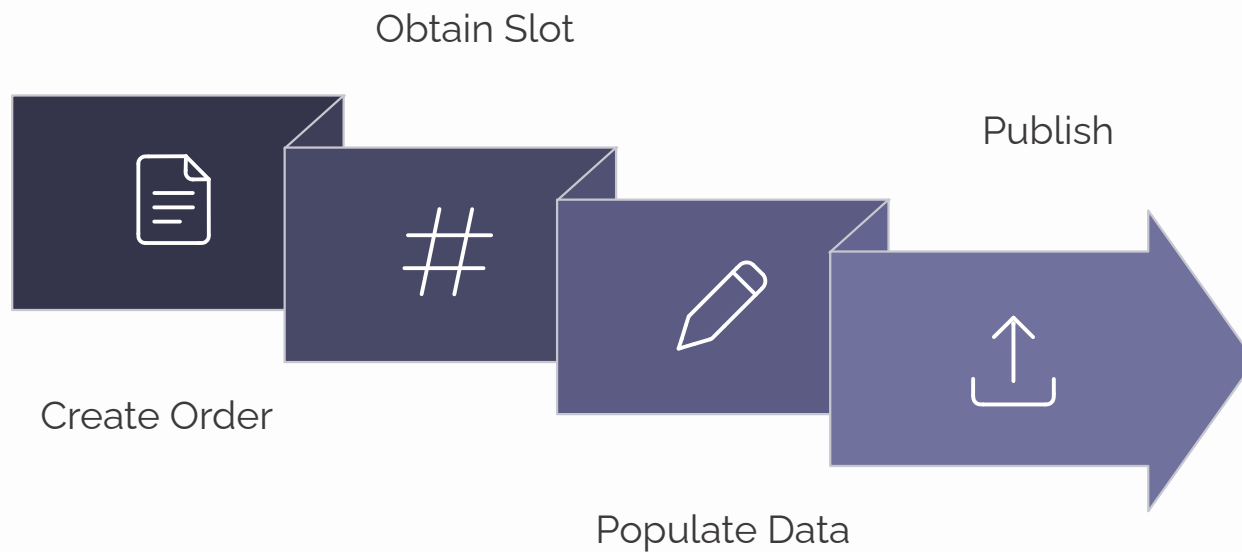
## Request Handling

Receives HTTP requests, validates input parameters, creates events, and returns structured responses.

## Error Handling

Validates order parameters and returns appropriate HTTP status codes with descriptive error messages.

# LMAX Disruptor Event Processing



## Why LMAX Disruptor?

Traditional locks and mutexes introduce contention and context switching. The Disruptor eliminates these completely.

### Ring Buffer

Pre-allocated circular buffer of fixed event slots — no garbage collection pressure, no allocation overhead.

### Lock-Free Design

Producers and consumers coordinate via sequence numbers — no mutexes, no blocking.

### Performance Benefit

High-throughput asynchronous processing without blocking HTTP request handlers.

# API Design and Integration

## REST Endpoints

| Method | Endpoint         | Purpose             |
|--------|------------------|---------------------|
| POST   | /api/orders      | Submit new order    |
| GET    | /api/orders      | Retrieve all orders |
| GET    | /api/orders/{id} | Get order details   |

## HTTP Status Codes

- **200 OK** — Successful processing
- **400 Bad Request** — Validation failure
- **500 Internal Server Error** — Processing failure

## Integration Flow

01

### Form Submission

User fills order form and submits via React frontend.

02

### HTTP POST Request

Axios sends JSON payload to Spring Boot REST controller.

03

### Service Processing

Service layer validates and publishes event to the Disruptor.

04

### Response Returned

Confirmation, status, and error details returned to the frontend.

# Testing and Validation

## API Testing — Postman

REST endpoints validated with varied payloads: valid orders, missing parameters, invalid inputs, and edge cases.

## Frontend Testing

Manual testing of UI components, form validation logic, and user interaction flows across the dashboard.

## Integration Testing

End-to-end validation of the full frontend → REST API → event processing → handler pipeline.

## Event Verification

Confirmed events are published to the ring buffer and consumed by handlers correctly without data loss.

## Error Scenarios

Tested invalid orders, duplicate submissions, and boundary conditions to ensure robust error handling.

# Team Contributions and Roles

| Name           | Role               | Responsibilities                                                                                        | Key Contributions                                                                                         |
|----------------|--------------------|---------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Tanvi Pandey   | Backend            | Designing Spring Boot REST APIs, service layer logic, and error handling for order processing.          | Developed core Spring Boot services and integrated the LMAX Disruptor for asynchronous order handling.    |
| Rama Sravanthi | Frontend           | Building the React dashboard UI, implementing order forms, and integrating with backend APIs.           | Crafted the responsive React dashboard and ensured seamless data flow between frontend and backend.       |
| Vishal         | Backend            | Implementing the LMAX Disruptor event processing, optimizing for high-throughput, and event management. | Configured the LMAX Disruptor ring buffer, significantly boosting event processing performance.           |
| Manish         | Coordination       | Project planning, team synchronization, managing requirements, and overseeing project documentation.    | Facilitated cross-functional team collaboration and ensured project deliverables were well-documented.    |
| Smiley         | Frontend / Testing | Frontend component development, manual UI testing, and comprehensive API validation using Postman.      | Authored critical frontend UI components and conducted thorough API testing to ensure system reliability. |

# Deployment and Maintenance Strategy



## Development Environment

Standardized local setup using IDEs like IntelliJ, with containerized databases (Docker Compose) and Maven/Gradle for dependency management.



## Production Deployment

Automated CI/CD pipelines (GitHub Actions) for Docker containerization and Kubernetes orchestration on AWS, enabling blue/green deployments.



## Maintenance Procedures

Regular patching, security updates, performance tuning, and scheduled code reviews to ensure system health and optimal performance.



## Version Control & Release Management

Git and GitHub for collaborative development, utilizing a Gitflow branching strategy with semantic versioning for releases and hotfixes.



## Testing Environment

Dedicated staging and UAT environments, mimicking production, with automated regression testing and secure data masking for sensitive information.



## Monitoring & Logging

ELK stack for centralized logging, integrated with Prometheus and Grafana for real-time application and infrastructure performance monitoring.



## Backup & Disaster Recovery

Automated daily database backups, configuration snapshots, and a comprehensive disaster recovery plan with defined RTO/RPO targets.

# Code Quality and Best Practices

Our development process integrates rigorous standards and methodologies to ensure high-quality, maintainable, and secure software.



## Code Review Process

Mandatory peer code reviews foster knowledge sharing, identify defects early, and maintain code consistency.



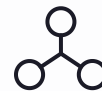
## Comprehensive Testing

Extensive unit, integration, and end-to-end tests guarantee functionality, stability, and adherence to requirements.



## Detailed Documentation

Clear and up-to-date documentation for APIs, architecture, and deployment procedures ensures smooth onboarding and maintenance.



## Design Patterns

Leveraging event-driven and producer-consumer patterns for scalable, resilient, and performant system architecture.



## Coding Conventions

Strict adherence to established style guides and best practices for readable, maintainable, and uniform code.



## Performance Optimization

Continuous profiling and optimization techniques applied to minimize latency and maximize throughput, especially with the LMAX Disruptor.



## Security Best Practices

Security integrated throughout the SDLC, following OWASP Top 10 guidelines to mitigate common vulnerabilities.



## Git Workflow

Utilizing a feature-branching workflow with pull requests and code merges for collaborative and controlled development.



# Risk Assessment and Mitigation

1

## Technical Risks

Potential performance bottlenecks or integration issues between frontend, backend, and the LMAX Disruptor.

**Mitigation:** Implement continuous load testing and performance profiling throughout development. Enforce strict API contract versioning and utilize mock services for early and isolated integration testing.

2

## Resource Risks

Key team members becoming unavailable due to illness or unforeseen circumstances, impacting project velocity.

**Mitigation:** Foster cross-functional knowledge sharing and document all critical components and processes. Maintain a well-prioritized backlog to allow for flexible task allocation and minimize single points of failure.

3

## Schedule Risks

Unexpected technical complexities or late changes in requirements causing project timeline delays.

**Mitigation:** Employ agile sprint planning with regular review meetings to adapt to changes. Incorporate contingency buffers in planning for critical phases and manage scope carefully through clear communication with stakeholders.

4

## Operational Risks

System reliability issues such as unexpected downtime, data loss, or security vulnerabilities in production.

**Mitigation:** Establish comprehensive monitoring and alerting systems for all services. Implement automated backups and conduct regular disaster recovery exercises. Follow security best practices and perform periodic vulnerability assessments.

# Performance Metrics and Results

Our order processing system achieves superior performance by leveraging a lock-free, high-throughput architecture. Below are the key metrics demonstrating its efficiency and speed, particularly when compared to traditional synchronous systems.



## High Throughput

Capable of processing **thousands of orders per second**, ensuring the system scales efficiently under heavy load and concurrent requests.



## Ultra-Low Latency

Achieves average response times in the **single-digit milliseconds** for critical order submission and retrieval operations, providing a real-time user experience.



## Optimized Resource Use

Maintains a **minimal memory footprint** and efficient CPU utilization by eliminating contention and reducing garbage collection overhead.



## Blazing Event Processing

The LMAX Disruptor enables events to be processed at **near wire speed**, decoupling API response times from the backend processing logic.

Unlike traditional synchronous systems that can suffer from bottlenecks due to locking mechanisms and context switching, our architecture, powered by the LMAX Disruptor, ensures non-blocking operations. This results in significantly higher throughput and lower, more predictable latency, making it ideal for high-frequency trading environments or any application demanding extreme performance.